Knowledge-Guided Semantics Adjustment for Improved Few-Shot Classification

Guangtao Zheng
Department of Computer Science
University of Virginia
gz5hp@virginia.edu

Aidong Zhang
Department of Computer Science
University of Virginia
aidong@virginia.edu

***Abstract*—In few-shot classification, each task usually contains a few examples with their labels carrying no semantic meanings, such as Class 1 or Class 2. Hence, it is challenging for deep neural networks to infer the true class of an image which contains multiple class-unrelated objects. In contrast, knowing what objects are of interest for constructing classification tasks, humans can identify the right class object from very few images from the same class and avoid misclassification on unlabeled images. Motivated by this observation, we aim to exploit prior knowledge from a given dataset and use it to filter out class-unrelated objects in a few-shot task. In our framework, we design a set of meta-learned semantic features with an importance kernel. Each feature corresponds to a common pattern found in many images. Our method first detects these semantic features in the given images and uses the importance kernel to suppress those identified as class-unrelated by the kernel so that the prediction model can focus on the true class objects of the images. To facilitate learning and identifying semantic features in an image, a simple and effective image representation decomposition module is further proposed to decouple complex correlations between objects in an image embedding. The experimental analysis demonstrates the effectiveness of our method, especially in the extremely low-shot cases.**

***Index Terms*—meta-learning, semantic features, few-shot classification**

I. Introduction

The ability of learning from few examples is attractive for deep learning models since it can greatly alleviate the need of the labor-intensive process of collecting many labeled examples for training. However, because of their large hypothesis space [1], widely used off-the-shelf deep learning models have difficulty in generalization when trained on few labeled examples. To systematically analyze and address this challenge, novel few-shot learning settings have been developed. Few-shot classification [2]–[6] is one example of such pursuit.

In few-shot classification, tasks are constructed such that each task consists of a support set and a query set. For each task, a model learns from the support set, which only has very few labeled examples per class, and predicts examples in the query set. In practice, the labels provided in each task, such as Class 1 and Class 2, carry no semantic meanings. Without any prior knowledge, it is challenging for a deep neural network to correctly infer the actual class from few images especially when they have multiple objects. Without substantial labeled data, the network may capture any of the objects or a combination of them from an image as the class object that distinguishes the image from other images in different classes. This is illustrated in Figure 1, where we fine-tune and evaluate a well-trained meta-learning model proposed in [7] on a challenging 2-way 1-shot testing task. The task is constructed with two classes representing two breeds of dogs, but the provided labels do not contain such information. Each class has one labeled image in the support set. We use class activation map (CAM) [8] as the visualization tool to show important areas contributing to predictions made by the model. For the support set images, we observe that in Class 2, the helmet, although being unrelated to the true class, is considered as important. As a result, the model tends to treat the task as distinguishing dogs with or without helmets, and thus the query image from Class 2 is misclassified as Class 1 since there is no helmet in the image.

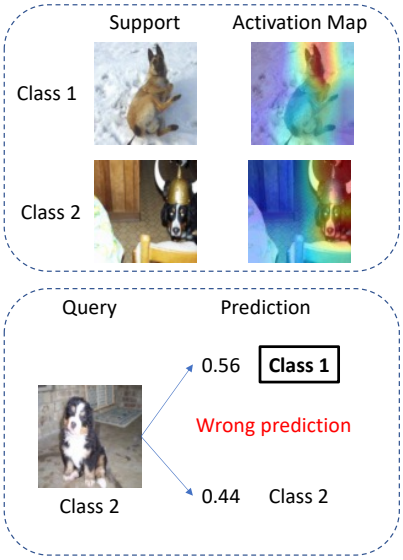


Fig. 1. Illustration on how class-unrelated objects in an image affects model prediction. Important areas contributing to the correct predictions made by the model are highlighted by CAM [8] with warmer colors representing higher importance. The helmet is considered important by the model for correctly recognizing Class 2 but it is unrelated to dogs.

It should be noted that even for humans, without any prior knowledge or context about the task, there is still a possibility that the query image is considered as Class 1. However, with the prior knowledge that for many tasks, dogs are more

probable to be an object of interest than other objects in an image, we tend to treat this task as to classify two breeds of dogs, and we will focus on the dogs instead of the helmet when making predictions since the helmet is not of interest according to our prior knowledge. To mimic such human-level behavior in the existence of prior knowledge to improve the performance of a few-shot classifier, we propose to learn semantically meaningful features along with their relative importance as the prior knowledge in few-shot classification. These features represent common patterns found in many images from many few-shot tasks, and we use them by first detecting the strengths of these features in each image and adjusting the strengths to suppress unimportant ones which are not of interest as compared to others.

More specifically, we propose a knowledge-guided semantics adjustment module (KGSA), which contains a set of learnable semantic features and a learnable importance kernel. Each of the semantic features represents a common pattern found in many images, such as the shape of dogs. The KGSA detects these semantic features in each image embedding and uses the importance kernel to determine which features are to be suppressed. Moreover, since an image embedding is obtained from a nonlinear feature extractor, it contains complex correlations between the objects in the input image, hindering the ability of KGSA in detecting and suppressing class-unrelated patterns in the image. To circumvent this, we propose a simple and effective sampling-based image representation decomposition (IRD) module to mitigate the coupling between objects in image embeddings. With the incorporation of IRD, class-unrelated patterns can be effectively detected and suppressed by KGSA in an image embedding, and class-related patterns can be captured by the model to infer the true class of the image.

In summary, the main contributions of our paper are:

- We propose to learn the prior knowledge on important objects for object recognition in few-shot classification by proposing a KGSA which consists of a set of semantic features and an importance kernel. The KGSA can detect interested object patterns using the semantic features and suppress class-unrelated ones based on the importance kernel to help a model infer the class of a given image.
- We propose a simple and effective IRD module to improve the effectiveness of the KGSA by mitigating the coupling between objects in an image embedding.
- We meta-train the KGSA on many randomly sampled few-shot classification tasks for improved few-shot classification performance. We analyze and validate the effectiveness of our method on two benchmark datasets [5], [9]. Our method achieves the best few-shot classification performance among all the relevant state-of-the-art approaches and shows superiority in dealing with extremely low-shot tasks.

II. Related Work

Many few-shot classification algorithms are based on meta-learning which captures from a large number of meta-training tasks the transferable knowledge on how to adapt to an unseen task. A task contains a support set and a query set, and each class in the support set usually has few labeled samples. Meta-learning uses the two sets in its inner loop and outer loop learning procedures. In the inner loop, meta-learning adapts a model's parameters using the support set data. Then, it evaluates and updates the model using the query set data. Depending on the specific learning method in the inner loop, many meta-learning algorithms can be divided into optimization-based and metric-based methods.

Optimization-based methods [10]–[15] use gradient descent to update part or all model parameters in the inner loop, and the goal of these methods is to learn a good model initialization that can be fast adapted to a new task using few gradient descent steps. In contrast, metric based methods [3], [4], [6], [7], [16]–[21] learn a shared embedding network for different tasks with a specialized classifier using various metric functions, such as MetaOptNet [22] using an SVM classifier, ProtoNet [4] using a prototype-based classifier with Euclidean distance, DeepEMD using the Earth Mover's Distance [18], or the method [20] using subspace distance. Our method belongs to the metric-based category, and the difference from existing methods is that we explicitly specify how to utilize the prior knowledge on few-shot classification tasks by proposing a novel KGSA module that regularizes few-shot classification with the prior knowledge encoded in it.

Moreover, using a set of learned features in KGSA is similar to several works in zero-shot [23] and the long-tailed recognition problem [24]. In [23], a set of virtual classifier weights are learned and combined to synthesize a classifier for real-world objects, with the combination weights calculated from external class attributes or class name embeddings. While in our work, the semantic features are obtained from training data only instead of from external data, and these features are used to adjust instead of generating classifier weights. Additionally, the semantic features and the importance kernel in KGSA are similar to the memory and the shallow network used in [24], where the memory features are linearly combined using the weights generated by the shallow network, and the summation is added to an image embedding. The key difference is the memory features in [24] are class centroids, while our semantic features are meta-learned and are not related to a specific class.

Our work also differs from the works using transductive learning, where a model uses all or part of the test samples before making predictions [25], such as performing label propagation through the union of the support and query sets [26], or conditioning class prototypes on a query sample [27]. In contrast, our work focuses on the inductive setting, where a prediction model for a task is only learned from the support set.

III. Problem Definition

In few-shot classification, each few-shot task $\mathcal{T}$ contains a support set $\mathcal{S} = \{(\mathbf{x}_i, z_i)|i = 1, \ldots, n_S\}$ and a query set $\mathcal{Q} = \{(\mathbf{x}_j, z_j)|j = 1, \ldots, n_Q\}$, where $\mathbf{x}_i$ and $\mathbf{x}_j$ are the examples,

and z_i and z_j are the corresponding labels. n_S denotes the size of the support set, and n_Q denotes the size of the query set. Typically, the support set contains N-way K-shot examples, i.e., there are N classes with K examples per class in the set; hence, $n_S = N \cdot K$. Similarly, there are the same N classes as in $\mathcal{S}$ with K_q examples per class in $\mathcal{Q}$, and $n_Q = N \cdot K_q$.

A model in few-shot classification is usually described as an embedding network $f_{\boldsymbol{\theta}}(\cdot)$ parameterized by $\boldsymbol{\theta}$ with a classifier $\ell_{\boldsymbol{\phi}}(\cdot)$ parameterized by $\boldsymbol{\phi}$. The embedding network maps an example $\mathbf{x}$ to its embedding $\mathbf{e} = f_{\boldsymbol{\theta}}(\mathbf{x}) \in \mathbb{R}^{D\times 1}$ with D denoting the embedding dimension. Then, the classifier takes $\mathbf{e}$ as its input and outputs the probability of $\mathbf{e}$ in each class. The objective in few-shot classification is to learn a model that minimizes the classification loss over a distribution of tasks $p(\mathcal{T})$, and it is formulated as follows:

$$\min_{\boldsymbol{\theta},\boldsymbol{\phi}} \quad \mathbb{E}_{\mathcal{T}\sim p(\mathcal{T})} \sum_{(\mathbf{x}_i,z_i)\in\mathcal{Q}} \mathcal{L}(\ell_{\boldsymbol{\phi}^*}(f_{\boldsymbol{\theta}^*}(\mathbf{x}_i)), z_i) \tag{1}$$

$$\text{s.t.} \quad \mathcal{T} = \{\mathcal{S}, \mathcal{Q}\}, \tag{2}$$

$$\boldsymbol{\theta}^*, \boldsymbol{\phi}^* = A(\boldsymbol{\theta}, \boldsymbol{\phi}, \mathcal{S}), \tag{3}$$

where $\mathcal{L}$ is the cross-entropy loss, and A represents a learning algorithm which uses $\mathcal{S}$, $\boldsymbol{\theta}$ and $\boldsymbol{\phi}$ as inputs and outputs task-specific weights, $\boldsymbol{\theta}^*$ and $\boldsymbol{\phi}^*$. Unlike the transductive setting, where $\boldsymbol{\theta}^*, \boldsymbol{\phi}^* = A(\boldsymbol{\theta}, \boldsymbol{\phi}, \mathcal{S}, \mathcal{Q})$, we focus on the inductive learning setting since $\boldsymbol{\theta}^*$ and $\boldsymbol{\phi}^*$ only depend on $\mathcal{S}$. Moreover, following the common setting in meta-learning [10], [18], [19], we assume the task distributions in meta-training and meta-testing are similar, i.e., tasks are all sampled from the same dataset such that knowledge transfer is possible.

The challenges in few-shot classification are the small size of the support set $\mathcal{S}$ and the lack of proper context since the provided labels carry no semantic meanings. Without enough data, $\boldsymbol{\theta}^*$ and $\boldsymbol{\phi}^*$ tend to fit to $\mathcal{S}$ with spurious patterns. For example, if there are multiple objects in an image, then the model may infer the wrong class for the image by capturing class-unrelated objects and still fits well for the examples in $\mathcal{S}$. As illustrated in Figure 1, Class 2 may be inferred as any dog with a helmet, but it actually represents a specific breed of dogs. Hence, the fitted model will generalize poorly on the query set $\mathcal{Q}$ since the wrong class is inferred.

IV. Methodology

In this section, we first give an overview of our proposed method and explain the novel components, the IRD and KGSA, in our method. Then, we describe how to learn a KGSA through meta-training.

A. Method Overview

Our method is motivated from the fact that even when there are multiple objects in an image, humans can still identify the one that represents the true class of the image due to the prior knowledge on important objects for object recognition. With the prior knowledge, we know which object is the most probable one to be the object of interest among all the objects in an image. Following this intuition, we aim to build a knowledge-guided module which resembles how we utilize prior knowledge for object recognition. The module encoded with proper prior knowledge can detect and adjust the strengths of certain patterns in each image such that class-unrelated patterns can be suppressed in the image embedding, and the true class of the image can be inferred.

Figure 2 illustrates the general framework of the proposed method which has two major components: IRD and KGSA. KGSA detects the strengths of certain patterns represented by the semantic features, and it adjusts the strengths of these semantic features to get the final embedding of an image such that class-unrelated patterns in the image are suppressed. IRD is introduced to improve the effectiveness of KGSA by decomposing an image embedding into a set of embeddings. Each embedding represents an area of interest of the image, and each area contains an object in the image. In this way, we decouple complex correlations between objects due to the nonlinearity of the embedding network, facilitating KGSA to detect object patterns and assign correct strengths to them in the image embedding. The set of adjusted embeddings processed by KGSA will be averaged to get the final embedding. In the following sections, we will explain each component in detail.

B. Image Representation Decomposition (IRD)

Due to the nonlinearity of the embedding network, multiple objects in an image have complex correlations in the image embedding, making it difficult to detect and suppress class-unrelated object patterns. To address this challenge, we introduce an IRD module to decompose an image embedding into a set of embeddings, each of which represents an image patch from the same input image. Ideally, each image patch represents a region of interest, containing an object to be recognized by the KGSA. In this way, we can decouple complex correlations between those objects, and the KGSA can effectively suppress class-unrelated object patterns to help a model detect the true class of the image.

However, in practice, it is not easy to find regions of interest from an image such that each region contains a meaningful object. We could use region proposal networks [28], [29] to find such regions, but training them in the few-shot setting is a separate and challenging problem. Therefore, in this paper, we design a simple random sampling approach in the IRD module and leave more powerful implementations of the IRD module as our future work.

Specifically, we randomly sample M regions, $\{\mathbf{r}_i | i = 1, \ldots, M\}$, from a given image $\mathbf{x}$ and resize them to $\{\tilde{\mathbf{r}}_i | i = 1, \ldots, M\}$ with the same size as $\mathbf{x}$. Then the new embedding for the image $\mathbf{x}$ is

$$h^M_{\boldsymbol{\theta}}(\mathbf{x}) = [f_{\boldsymbol{\theta}}(\tilde{\mathbf{r}}_1), \ldots, f_{\boldsymbol{\theta}}(\tilde{\mathbf{r}}_M)], \tag{4}$$

where $h^M_{\boldsymbol{\theta}}(\mathbf{x}) \in \mathbb{R}^{D\times M}$ is the embedding matrix for the input $\mathbf{x}$. In the trivial case when $M = 1$, the IRD will output the original image embedding $f_{\boldsymbol{\theta}}(\mathbf{x})$.

Note that we may miss or cut out some objects in an image using the random sampling strategy. However, this is not

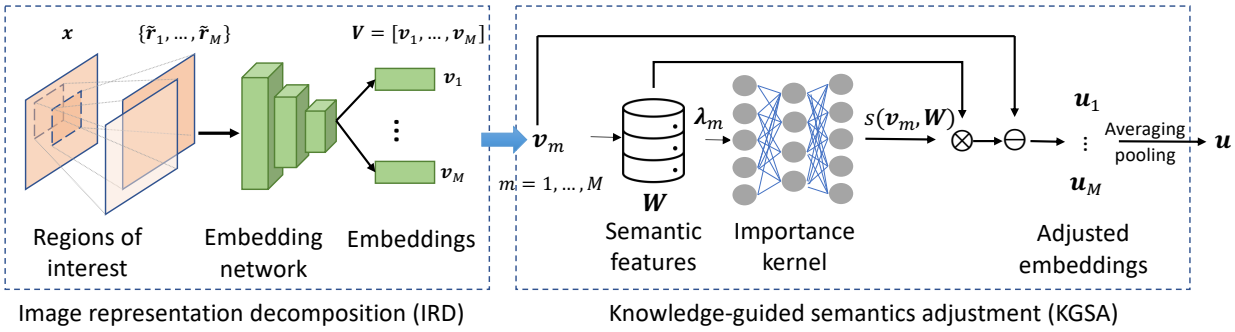


Fig. 2. The overview of our proposed method. Our method includes IRD and KGSA as the two key components.

critical in both training and testing phases. As in the training phase, the imperfectly sampled image patches that contain only part of the objects will act as a regularization which avoids the model remembering specific objects. In the testing phase, we can sample more image patches than those in the training to ensure sufficient coverage of all meaningful objects in the image. Although the sampling method used in IRD is often used for data augmentation, in our method, it serves as a simple and effective method to decouple complex relations between objects and to improve the effectiveness of the KGSA which will be detailed below.

C. Knowledge-Guided Semantics Adjustment (KGSA)

KGSA consists of a set of learnable semantic features $\mathbf{W}$ and a learnable importance kernel $\kappa_{\boldsymbol{\psi}}(\cdot)$ with parameters $\boldsymbol{\psi}$. Each of the semantic features corresponds to a common object pattern across images. For example, some features may correspond to the objects that are noticeable in certain images but are often not of interest in object recognition; some features may correspond to straight lines or curves that should be maintained in certain images to make them discriminative. The importance kernel is a learnable function that encodes the relative importance of the semantic features. It maps the strengths of the semantic features detected in an image embedding to the adjusted ones based on which the semantic features are either suppressed or preserved facilitating the inference of the true class of the image. KGSA is learned via meta-learning [4], [10], [19] due to its capability of capturing transferable knowledge across many few-shot tasks. In the following, we explain how KGSA works in few-shot classification and provide its training method in Section IV-D.

We denote the whole KGSA as $g_{\boldsymbol{\omega}}(\cdot)$ with $\boldsymbol{\omega} = \{\mathbf{W}, \boldsymbol{\psi}\}$. Then, for each $\mathbf{v}_m = h^M_{\boldsymbol{\theta}}(\mathbf{x})[m]$ from the m-th element of the IRD output $h^M_{\boldsymbol{\theta}}(\mathbf{x})$, KGSA outputs adjusted embedding $\mathbf{u}_m$ for each $\mathbf{v}_m$ by subtracting from $\mathbf{v}_m$ the semantic features $\mathbf{W} = [\mathbf{w}_1, \ldots, \mathbf{w}_L] \in \mathbb{R}^{D\times L}$ with certain suppression weights, i.e.,

$$\mathbf{u}_m = g_{\boldsymbol{\omega}}(\mathbf{v}_m) = \mathbf{v}_m - \mathbf{W}s(\mathbf{v}_m, \mathbf{W}), m = 1, \ldots, M \quad (5)$$

where $s(\mathbf{v}_m, \mathbf{W}) \in \mathbb{R}^{L\times 1}$ denotes the suppression weight vector dependent on $\mathbf{v}_m$ and $\mathbf{W}$. Finally, all the M adjusted vectors $\{\mathbf{u}_m | m = 1, \ldots, M\}$ are averaged to get the adjusted embedding $\mathbf{u}$ for the input $\mathbf{x}$, i.e.,

$$\mathbf{u} = 1/M \cdot \sum_{m=1}^{M} \mathbf{u}_m. \quad (6)$$

In our design of KGSA, calculation of the weights follows the *detection-and-adjustment* procedure due to its flexibility in imposing constraints on the semantic features and its low computational complexity.

1) Detection.: KGSA first detects the strengths of semantic features $\mathbf{W}$ in the embedding $\mathbf{v}_m$ by calculating the strength vector $\boldsymbol{\lambda}_m = [\lambda_{1,m}, \ldots, \lambda_{L,m}]^T$, a sparse decomposition of $\mathbf{v}_m$ in terms of $\mathbf{W}$. Specifically, $\boldsymbol{\lambda}_m$ is obtained via solving the following problem:

$$\boldsymbol{\lambda}_m = \arg\min_{\boldsymbol{\lambda}} \|\mathbf{W}\boldsymbol{\lambda} - \mathbf{v}_m\|_2^2 + \gamma\|\boldsymbol{\lambda}\|_2^2, \quad (7)$$

where γ is a non-negative hyperparameter controlling the magnitude of $\boldsymbol{\lambda}_m$, i.e., $\|\boldsymbol{\lambda}_m\|_2$. The solution $\boldsymbol{\lambda}_m$ quantifies the similarities of these semantic features to the embedding $\mathbf{v}_m$. The advantage of introducing Eq. (7) is that we can impose an explicit sparse constraint on $\boldsymbol{\lambda}_m$ and consequently enforce these semantic features to be as general as possible to cover many similar objects. Moreover, we can get the closed form solution for $\boldsymbol{\lambda}_m$ as $\boldsymbol{\lambda}_m = (\mathbf{W}^T\mathbf{W} + \gamma\mathbf{I})^{-1}\mathbf{W}^T\mathbf{v}_m$, where $\mathbf{I} \in \mathbb{R}^{L\times L}$ is an identity matrix. Note that γ not only controls the magnitude of $\boldsymbol{\lambda}$, hence the representation capability of $\mathbf{W}$, but also makes sure the inverse operation is not ill-conditioned. In practice, we find that our method is not sensitive to γ in terms of few-shot classification performance, so we set $\gamma = 1$.

2) Adjustment.: Then, KGSA adjusts $\boldsymbol{\lambda}_m$ using its importance kernel $\kappa_{\boldsymbol{\psi}}(\cdot)$ which is an L-input L-output function to get $s(\mathbf{v}_m, \mathbf{W})$. Using the importance kernel is critical since KGSA could detect two semantic features that have large λs and needs to determine which one should be suppressed. For example, if the two semantic features $\mathbf{w}_1$ and $\mathbf{w}_2$ correspond to head covering and the shape of dogs, respectively, then we expect to suppress $\mathbf{w}_1$ while preserving $\mathbf{w}_2$ for images in Figure 1 since we need to focus on dogs according to the prior knowledge. Hence, we need the importance kernel to learn the relative importance of all the semantic features such that only class-unrelated ones are suppressed. In practice, the

importance kernel is implemented with an L-input L-output neural network, so the output may contain negative numbers. Subtracting features with negative weights will strengthen them in $\mathbf{v}_m$ and will blur the class-specific information in $\mathbf{v}_m$ since the semantic features are not designed to be class-specific. Hence, the suppression weight vector is calculated as

$$s(\mathbf{v}_m, \mathbf{W}) = \max(\kappa_{\boldsymbol{\psi}}(\boldsymbol{\lambda}_m), \mathbf{0}), \tag{8}$$

where $\max(\cdot,\cdot)$ is an element-wise maximum operator, and $\mathbf{0}$ is a length-L all-zero column vector.

D. Meta-Learning KGSA

In essence, KGSA is designed to highlight class-related objects in a dataset by suppressing unrelated ones so that the few-shot classification performance of a model on the dataset can be improved. Hence, we forward the outputs from the KGSA module to a few-shot classifier to optimize the classification performance and tune $\boldsymbol{\omega}$ accordingly. Specifically, we use the adjusted embedding vectors of the support set data to train the few-shot classifier which is a prototype-based, non-parametric, and nearest-neighbor classifier used in many few-shot classification literature [4], [7], [17], [19], [30]. Learning a task-specific classifier is simply to calculate the class prototypes [4] from the support set of the task. In an N-way K-shot task, each of the N class prototypes is calculated as follows,

$$\boldsymbol{\mu}_c = \frac{1}{|\mathcal{F}_c|} \sum_{\mathbf{u} \in \mathcal{F}_c} \mathbf{u}, c = 1, \ldots, N \tag{9}$$

where $|\cdot|$ denotes the size of a set, $\mathcal{F}_c = \{\mathbf{u} | \mathbf{u} = 1/M \sum_{m=1}^{M} g_{\boldsymbol{\omega}}(h_{\boldsymbol{\theta}}^{M}(\mathbf{x}_s)[m]), \mathbf{x}_s \in \mathcal{S}_c\}$, and $\mathcal{S}_c$ is the set of support examples from class c. The N class prototypes will instantiate a prototype-based classifier $p(\cdot|\mathbf{x}_q, \boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_N)$ whose prediction for a query example $\mathbf{x}_q \in \mathcal{Q}$ is

$$\hat{c} = \arg \max_{c=1,\ldots,N} p(y = c|\mathbf{x}_q, \boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_N), \tag{10}$$

where $p(y = c|\mathbf{x}_q, \boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_N)$ is defined as follows,

$$p(y = c|\mathbf{x}_q, \boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_N) = \frac{\exp(\tau d(\mathbf{u}_q, \boldsymbol{\mu}_c))}{\sum_{c'} \exp(\tau d(\mathbf{u}_q, \boldsymbol{\mu}_{c'}))}, \tag{11}$$

where $d(\cdot,\cdot)$ denotes a similarity metric function; $\boldsymbol{u}_q$ is the embedding for the query example $\mathbf{x}_q$ obtained from the KGSA; τ controls the softness of the output probability distribution, or equivalently, controls how much a classification error will be penalized. In this paper, we use cosine similarity as the metric function thanks to the good generalization performance it brings to few-shot classification [7], [31], and we choose τ via hyperparameter selection.

Next, we follow the metric-based meta-training method [3], [4], [7], [19], which can efficiently extract shared knowledge across different tasks, to train the whole model, including the training of KGSA. Specifically, we minimize the risk function $\mathcal{L} = \mathbb{E}_{\mathcal{T} \sim p(\mathcal{T})} \mathcal{L}_{\mathcal{T}}$ with respect to $\boldsymbol{\theta}$ and $\boldsymbol{\omega}$ as follows,

$$\boldsymbol{\theta}^*, \boldsymbol{\omega}^* = \arg \min_{\boldsymbol{\theta}, \boldsymbol{\omega}} \mathbf{E}_{\mathcal{T} \sim p(\mathcal{T})} \mathcal{L}_{\mathcal{T}}, \tag{12}$$

where $p(\mathcal{T})$ is the task sampling distribution, and the individual task loss $\mathcal{L}_{\mathcal{T}}$ is defined as

$$\mathcal{L}_{\mathcal{T}} = -\frac{1}{|\mathcal{Q}|} \sum_{(\mathbf{x}, z) \in \mathcal{Q}} \log p(y = z|\mathbf{x}, \boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_N), \tag{13}$$

where $\mathcal{T} = \{\mathcal{S}, \mathcal{Q}\}$, z is the label for the example $\mathbf{x}$, and $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_N$ are obtained from the support set $\mathcal{S}$ following Eq. (9). The true risk $\mathcal{L}$ is hard to obtain since $p(\mathcal{T})$ is unknown; hence, we calculate the empirical risk of B tasks as $\mathcal{L}_E = \frac{1}{B} \sum_{b=1}^{B} \mathcal{L}_{\mathcal{T}_b}$ with $\mathcal{T}_b$ sampled from the training data. Then, we use gradient descent with a learning rate β to minimize $\mathcal{L}_E$ each time after sampling B tasks as $(\boldsymbol{\theta}, \boldsymbol{\omega}) = (\boldsymbol{\theta}, \boldsymbol{\omega}) - \beta \nabla_{\boldsymbol{\theta}, \boldsymbol{\omega}} \mathcal{L}_E$. The above iteration process will terminate when the classification accuracy on a validation set stops increasing. Note that due to our specific implementation of the IRD module, no learnable parameters are required for this module. The procedures for meta-training KGSA and inference with KGSA are shown in Algorithm 1 and Algorithm 2, respectively.

In our method, two important hyperparameters affect the effectiveness of KGSA: the number of sampled regions M in IRD and the number of semantic features L in KGSA. With a larger M, IRD can provide more accurate object information to KGSA. However, a larger M will take more time and memory to run the optimization in Eq. (12). The effectiveness of KGSA on many unseen tasks also depends on L. For a small and simple dataset which contains a small number of classes, the KGSA with a large L will overfit to the dataset, capturing features corresponding many specific objects. On the other hand, for a large dataset with many classes, we expect L to be larger. In practice, we will use a held-out validation set to select the best values of L.

Algorithm 1 Meta-Training KGSA

Input: Task distribution $p(\mathcal{T})$, learning rate β, number of sampled regions M, number of semantic features L
Initialize $\boldsymbol{\theta}$, $\boldsymbol{\omega} = \{\mathbf{W}, \boldsymbol{\psi}\}$
while Not done **do**
  Sample a batch of B meta-training tasks $\{\mathcal{T}_b | b = 1, \ldots, B\}$ from $p(\mathcal{T})$
  Calculate $\mathcal{L}_E = \frac{1}{B} \sum_{b=1}^{B} \mathcal{L}_{\mathcal{T}_b}$ using Eq. (13)
  Update $(\boldsymbol{\theta}, \boldsymbol{\omega}) = (\boldsymbol{\theta}, \boldsymbol{\omega}) - \beta \nabla_{\boldsymbol{\theta}, \boldsymbol{\omega}} \mathcal{L}_E$
end while
Output: $\boldsymbol{\theta}$, $\boldsymbol{\omega}$

V. Experiment

In this section, we first describe two datasets used in our experiments and give important implementation details of our method. Next, we compare our method with the state-of-the-art methods on these datasets. Afterwards, we analyze how different designs of the KGSA affect the few-shot classification performance in our ablation studies. Then, we analyze the function of a meta-learned importance kernel. Finally, we will provide an example with visualizations to demonstrate how our method works.

Algorithm 2 Inference with KGSA

1: **Input**: A meta-testing task $\mathcal{T} = \{\mathcal{S}, \mathcal{Q}\}$, number of sampled regions M'
2: Randomly sample regions with M' according to Eq. (4) for all examples in $\mathcal{S}$ and $\mathcal{Q}$
3: Obtain adjusted embeddings according to Eq. (5) and Eq. (6)
4: Obtain adjusted classifier weights according to Eq. (9)
5: Obtain predictions for query examples $\hat{c}_{\mathcal{Q}} = \{\hat{c}_q | q = 1, \ldots, n_{\mathcal{Q}}\}$ according to Eq. (10)
6: **Output**: $\hat{c}_{\mathcal{Q}}$

A. Datasets

a) miniImageNet.: This dataset [9] contains 100 classes chosen randomly from ILSVRC-2012 [32], each having 600 images. Each image is reshaped to have the size of 84×84. We follow the dataset split from [9] and divide the dataset into three non-overlapping sets of classes, forming the meta-train, meta-validation, and meta-test splits. The number of classes in the three splits are 64, 16, and 20, respectively.

b) tieredImageNet.: This dataset [5] is a large subset of ILSVRC-2012 [32], consisting of 608 classes. These classes belong to 34 semantic categories. Within these semantic categories, 20 of them are allocated for meta-training, 6 for meta-validation, and 8 for meta-testing. This results in 351, 97, and 160 classes for meta-training, meta-validation, and meta-testing, respectively. All images are resized to 84×84.

B. Implementation Details

Training protocols. We adopt the ResNet-12 used in [18], [19], [33] as the embedding network. To provide a good initialization for meta-training, we first pre-train the embedding network using the SGD optimizer with 0.9 momentum, 0.1 initial learning rate, and 5×10^{-4} weight decay. Then, we meta-train the whole model using the same SGD optimizer but with different initial learning rates. We set the initial learning rate to 5×10^{-4} and 1×10^{-5} for the miniImageNet and the tieredImageNet datasets, respectively. The model that achieves the best validation accuracy will be used for evaluation.

Model configurations. The importance kernel in KGSA is implemented as a feed-forward neural network with one hidden layer having H units and ReLU non-linear activations. Dropout [34] is applied to the output of the importance kernel to prevent the semantic features from overfitting. The KGSA module can be configured with L and H. The best configuration for KGSA is found via hyperparameter search using the accuracy on the validation set. For 1-shot experiments on the miniImageNet, the KGSA is configured as $(L, H) = (10, 50)$, and switched to $(L, H) = (5, 50)$ for 5-shot experiments. The configurations change to $(L, H) = (100, 50)$ and $(L, H) = (50, 50)$ for 1- and 5-shot experiments on the tieredImageNet, respectively. *Unless otherwise stated, the sampling number M in IRD is set to 5 in training and 50 in evaluation.*

Evaluation. All the experiments are conducted in the inductive setting where no query data are involved in generating the prediction model. The statistics in the batch normalization layers are fixed in the evaluation to avoid knowledge sharing between tasks. We sample an N-way K-shot task by first sampling N classes randomly from all the classes in the test split of a dataset, and further randomly sample support and query samples for each of the N classes. We report the mean accuracy and the standard deviation with a 95% confidence interval over 2000 randomly sampled few-shot tasks.

Our code is implemented with PyTorch [35]. All the experiments are conducted on the Nvidia RTX 8000 GPUs.

C. Results on Benchmark Datasets

Table I shows the 5-way classification accuracy comparison between our method and existing few-shot learning methods. For fair comparison, we only consider methods using inductive learning and exclude methods using transductive learning. We also exclude methods exploring self-supervised learning [41], [42] since our method does not use additional unlabeled data. We observe from Table I that our method achieves the best performance among all the listed baseline methods in both 1- and 5-shot settings. Specifically, on miniImageNet, our method achieves 2.88% and 2.05% accuracy increment compared with the competitive method, CPLAE, in 1-shot and 5-shot settings, respectively. On tieredImageNet, our method achieves 2.07% and 0.23% accuracy increment compared with CPLAE in the 1-shot and 5-shot settings, respectively. Overall, our method has the best performance in all evaluation settings with significant performance gains achieved in 1-shot settings. Moreover, from the perspective of additionally added parameters, the overhead of our method is negligible compared to the embedding network, since according to our experiment setting, the number of added parameters equals $L \times D + 2L \times H$, which is the combination of numbers from the semantic features and the importance kernel in KGSA.

D. Ablation Studies

1) Contributions of IRD and KGSA: We analyze how IRD and KGSA contribute to the few-shot classification accuracy. According to the discussion in Section IV-D, we set M to 5 in both training and testing to speed up evaluation and to best showcase the advantage of KGSA when low quality image embeddings are provided by the IRD with small M. We denote the model [7] whose embedding network is pre-trained and then fine-tuned with a few-shot classifier as the "Base" model. Unlike [7], we follow the setting in [18], [19] to obtain the "Base" model which has slightly better performance. Then, we add a KGSA module to "Base" and get "Base+KGSA". We also add to "Base" an IRD module and call the combination as "Base+IRD". Finally, we add both the KGSA and the IRD modules to "Base" and obtain "Base+KGSA+IRD". From the results in Table II, we observe that with KGSA alone, the performance of "Base+KGSA" is on par with some recent works, e.g., FEAT [19] and DeepEMD [18]. Adding IRD can improve the performance of "Base" and "Base+KGSA", with a bigger improvement from combining IRD with "Base+KGSA". For example, on the

TABLE I

PERFORMANCE COMPARISON TO PRIOR WORKS ON THE MINIIMAGENET AND TIEREDIMAGENET DATASETS. 5-WAY CLASSIFICATION ACCURACY (%) WITH 95% CONFIDENCE INTERVAL IS REPORTED. RESULTS WITH CITATIONS ARE REPORTED FROM THE LITERATURE WITH "-" REPRESENTING "NOT REPORTED". THE BEST PERFORMANCE IS IN THE BOLD FONT.

Method	Backbone	miniImageNet		tieredImageNet	
		1-shot	5-shot	1-shot	5-shot
TADAM [17]	ResNet-12	58.50 ± 0.30	76.70 ± 0.30	-	-
ProtoNet [4]	ResNet-12	60.37 ± 0.83	78.02 ± 0.57	65.65 ± 0.92	83.40 ± 0.65
Shot-Free [36]	ResNet-12	59.04 ± 0.43	77.64 ± 0.39	66.87 ± 0.43	82.64 ± 0.43
LEO [13]	WRN-28-10	61.76 ± 0.08	77.59 ± 0.10	66.33 ± 0.05	81.44 ± 0.09
MetaOptNet [22]	ResNet-12	62.64 ± 0.61	78.63 ± 0.46	65.99 ± 0.72	81.56 ± 0.53
CAN [37]	ResNet-12	63.85 ± 0.48	79.44 ± 0.34	69.89 ± 0.51	84.23 ± 0.37
MTL [38]	ResNet-12	61.20 ± 1.80	75.50 ± 0.80	65.62 ± 1.80	80.61 ± 0.9
Meta-Baseline [7]	ResNet-12	63.17 ± 0.23	79.26 ± 0.17	68.62 ± 0.27	83.29 ± 0.18
Neg-Cosine [39]	ResNet-12	63.85 ± 0.81	81.57 ± 0.56	-	-
DSN-MR [20]	ResNet-12	64.60 ± 0.72	79.51 ± 0.50	67.39 ± 0.82	82.85 ± 0.56
FEAT [19]	ResNet-12	66.78 ± 0.20	82.05 ± 0.14	70.80 ± 0.23	84.79 ± 0.16
DeepEMD [18]	ResNet-12	65.91 ± 0.82	82.41 ± 0.56	71.16 ± 0.87	86.03 ± 0.58
MELR [21]	ResNet-12	67.40 ± 0.43	83.40 ± 0.28	72.14 ± 0.51	87.01 ± 0.35
CPLAE [40]	ResNet-12	67.46 ± 0.44	83.22 ± 0.29	72.23 ± 0.50	87.35 ± 0.34
Our method	ResNet-12	**70.34±0.45**	**85.27±0.28**	**74.30±0.50**	**87.58±0.33**

TABLE II

ABLATION STUDY ON KGSA AND IRD ($M = 5$ IN BOTH TRAINING AND TESTING). 5-WAY CLASSIFICATION ACCURACIES WITH 95% CONFIDENCE INTERVALS ARE REPORTED.

Model	miniImageNet	
	1-shot	5-shot
Base	65.78 ± 0.46	82.07 ± 0.31
Base+KGSA	67.05 ± 0.45	82.64 ± 0.31
Base+IRD	67.34 ± 0.46	82.02 ± 0.31
Base+KGSA+IRD	68.67 ± 0.45	83.67 ± 0.30
Model	**tieredImageNet**	
	1-shot	5-shot
Base	71.76 ± 0.52	85.41 ± 0.35
Base+KGSA	71.88 ± 0.52	86.00 ± 0.35
Base+IRD	72.19 ± 0.51	85.60 ± 0.35
Base+KGSA+IRD	72.76 ± 0.52	86.74 ± 0.35

TABLE III

CLASSIFICATION ACCURACY OF THE "BASE+KGSA" MODEL CONFIGURED WITH DIFFERENT LS AND DIFFERENT METHODS FOR ADJUSTING STRENGTH VECTORS.

Adjusting Strength Vectors	Shot	L				
		1	5	10	50	100
Importance kernel	1-shot	66.42	66.85	67.05	66.97	66.62
	5-shot	82.29	82.64	82.55	82.54	82.34
Linear transform	1-shot	66.30	66.32	66.29	66.13	65.83
	5-shot	82.35	82.52	82.48	81.19	81.15

miniImageNet dataset, the performance gains for adding IRD to "Base" and "Base+KGSA" in the 1-shot case are 1.56% and 1.62%, respectively; for the 5-shot case, IRD can improve the accuracy of "Base+KGSA" by 1.03%, while being ineffective for "Base".

2) Semantic Features and Importance Kernel in KGSA: We study how the number of semantic features L affects the effectiveness of KGSA and analyze the necessity of importance kernel in KGSA. Table III shows the classification accuracies of the "Base+KGSA" model on the miniImageNet dataset with $L \in \{1, 5, 10, 50, 100\}$ and two different methods of adjusting the strength vector defined in Eq. (7). The baseline is "Linear transform" which means we apply linear transformation separately to each strength value, while "Importance kernel" represents feed forward neural networks proposed in our method. We observe that models adopting importance kernels perform better than those adopting linear transforms as L increases; specifically, we see a significant performance drop on the latter when $L \in \{50, 100\}$. This is because an importance kernel is context dependent: it compares strength values of different semantic features in an image embedding and adjusts these values jointly instead of transforming them separately. Hence, an importance kernel is necessary in KGSA. We also observe that in different settings, the optimal L is different; a larger shot setting generally requires a smaller L. Moreover, models with $L = 1$ have competitive performance. However, it is not the result of introducing additional parameters since adding an additional layer to the embedding network will get inferior performance with classification accuracies on miniImageNet being 63.14 ± 0.46 and 79.57 ± 0.33 for 1-shot and 5-shot, respectively. Instead, when $L = 1$, we observe that the learned semantic feature represents many noisy background objects, and models can benefit by suppressing it in image embeddings.

3) Effect of Sampling Times in IRD: We compare the classification accuracy of "Base+IRD" with that of "Base+KGSA+IRD" when IRD is configured with different Ms. Since a higher M in training requires more GPU memory, we train the models with $M = 5$ and evaluate them with $M \in \{5, 10, 15, 25, 50\}$. Table IV shows classification accuracies on the miniImageNet dataset. We observe consistent performance improvement on both shot numbers as M increases, and the performance increment saturates when M is large. Moreover, we show performance gains of our method, "Base+KGSA+IRD", over the "Base+IRD" model in parentheses following the accuracy numbers. We observe that when M increases, the KGSA module benefits more from the IRD module than the "Base" model does, indicating the

TABLE IV
CLASSIFICATION ACCURACY COMPARISON BETWEEN "BASE+IRD" AND "BASE+KGSA+IRD" WITH IRD CONFIGURED WITH DIFFERENT MS ON 5-WAY 1-SHOT AND 5-WAY 5-SHOT TASKS SAMPLED FROM THE MINIIMAGENET DATASET.

M	Base+IRD		Base+KGSA+IRD	
	1-shot	5-shot	1-shot	5-shot
5	67.34	82.02	68.67 (+1.33)	83.67 (+1.65)
10	67.90	82.57	69.31 (+1.41)	84.43 (+1.86)
15	68.16	82.86	69.65 (+1.49)	84.59 (+1.73)
25	68.41	83.02	69.75 (+1.34)	84.86 (+1.84)
50	68.63	83.07	**70.34** (+1.71)	**85.27** (+2.20)

necessity of IRD in providing high-quality image embeddings for KGSA.

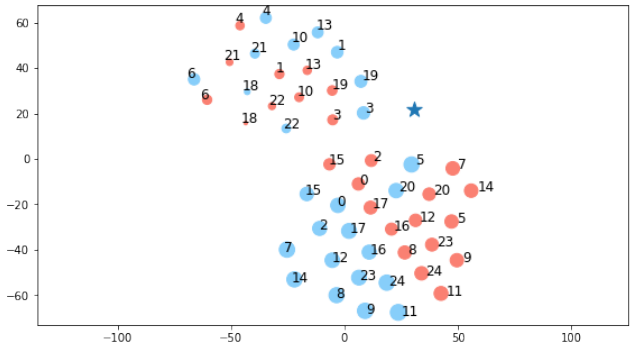
E. Analysis on the Importance Kernel

The importance kernel is designed to suppress class-unrelated features from an image embedding such that the adjusted embedding can be a better representation of its class than the original one. To demonstrate this, we analyze the importance kernel from a "Base+KGSA" model with $L = 100$ trained on the tieredImageNet dataset. We first select an image from Figure 1 and randomly sample 25 patches from the image as shown in Figure 3(a). This roughly divides the image into multiple meaningful parts, such as the helmet, the dog, and the background. Then, each of these patches is processed by the importance kernel separately, and we plot their embeddings before and after the kernel in Figure 3(b). The class embedding (marked as the star) is obtained by averaging all the sample embeddings from the same class in the same task. We observe a top and a bottom clusters from Figure 3(b). Points in the top cluster correspond to image patches enclosed with red squares in Figure 3(a); these patches are class-unrelated. The importance kernel moves these patch embeddings (blue points) away from the class embedding and reduces their magnitudes (shown as the smaller red points). Points in the bottom cluster correspond to image patches that contain the class-related object, i.e., the dog. The importance kernel moves these patch embeddings closer to the class embeddings while preserving their magnitudes as evidenced by the red points with their sizes similar to those of the blue points. In summary, the learned importance kernel effectively suppresses class-unrelated objects in an image and highlights class-related ones. The result shown in Figure 3(b) also justifies the use of Eq. (6) since the adjusted embeddings of class-unrelated image patches will have small magnitudes and the class-related ones can be highlighted in the output of KGSA.

F. An Example on How KGSA Works

We trained a "Base+KGSA" model with $L = 100$ on the tieredImageNet dataset. Then, for the task given in Figure 1, we analyzed the semantic features in the model in Figure 4a which shows images whose embeddings are close to the two semantic features selected from a total of 100 features in the model. The suppressed feature indicates the suppression weight for the feature is positive for all images in the task

(a) Image patches



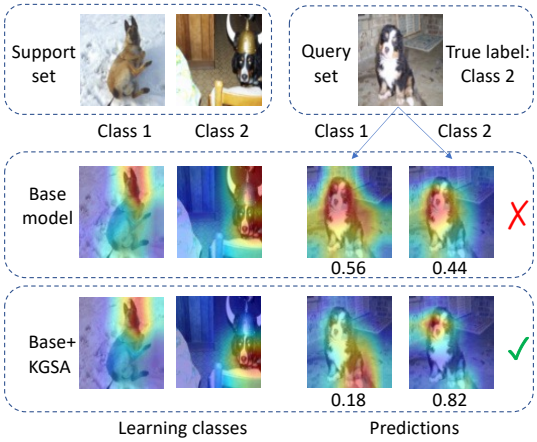
(b) Patch embeddings

Fig. 3. (a) 25 image patches sampled from an image labeled with Class 2 in Figure 1. The patches are labeled from 0 to 24 starting from the left to the right on the first row. (b) Visualization (using t-SNE [43]) of the image patch embeddings before (blue dots) and after (red dots) being processed by the importance kernel. The size of each point scales positively with the norm of its embedding. The star represents the embedding of Class 2. The points roughly lie in the top and the bottom clusters. The labels in the top cluster correspond to the patches enclosed with red squares in (a).

shown in Figure 4b, so the feature will be subtracted from all image embeddings from the task. In contrast, the preserved feature means the weight for the feature is zero, and the feature has no effect on the image embeddings. We see that objects in the images related to the suppressed feature all have their heads covered. Moreover, although from different classes, images related to the preserved feature in the lower box have similar colors and shapes. Thus, for this task, KGSA filters out the head covering feature and keeps potentially important objects untouched. This is because the head covering feature is detected from the image with a dog and a helmet, and KGSA learns from the training that suppressing this feature generally benefits few-shot classification.

In Figure 4b, we show how the model succeeds in making the correct prediction while the Base model fails for the task in Figure 1. For each image, we use CAM [8] to highlight important parts of the image when a model predicts it as Class 1 or Class 2. As analyzed in Section I, the Base model tends to distinguish dogs with or without helmets. In contrast, with the help of KGSA, the unrelated head covering feature is suppressed, and the "Base+KGSA" model can successfully

(a) Semantic features

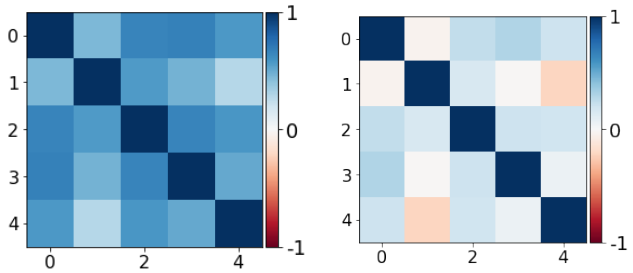


(b) CAM visualization

Fig. 4. Analysis of the "Base+KGSA" model with $L = 100$. (a) Images whose embeddings are close to each of the semantic features. (b) CAM visualization showing how the "Base+KGSA" model succeeds in making the correct prediction while the "Base" model fails. CAM highlights important areas with warm colors and unimportant areas with cool colors. The task is sampled from the tieredImageNet dataset.

associate Class 2 with the dog since in the corresponding CAM visualization, the helmet is covered with cool colors and thus is not considered as important. We also observe a more focused important area in "Base+KGSA" than the one for the Base model from the CAM visualizations of the query image when the two models predict it as Class 2. Hence, the prediction confidence on Class 2 increases from 0.44 to 0.82.

Furthermore, Figure 5 shows the pairwise cosine similarities between the class prototypes (defined in Eq. (9)) obtained before and after the KGSA in a 5-way task. The figure shows KGSA can make embeddings of different classes more distinctive than the original ones.



(a) Before KGSA (b) After KGSA

Fig. 5. Pairwise similarities between the class prototypes before and after KGSA.

VI. Conclusion

In this paper, we introduced a knowledge-guided semantics adjustment module (KGSA) which consists of a set of learnable semantic features and a learnable importance kernel. The semantic features represent common patterns shared by many similar objects. The importance kernel encodes relative importance of the features and can suppress the class-unrelated ones. The KGSA follows the detection-and-adjustment procedure to highlight class-related objects in image embeddings. We meta-trained the KGSA via optimizing the final few-shot classification performance. To facilitate the KGSA in detecting and suppressing class-unrelated features in an image embedding, we further proposed an image representation decomposition (IRD) module to decouple the potential correlations between objects in the image embedding due to the nonlinearity of the embedding network. Our method outperforms the state-of-the-art methods in two few-shot benchmark datasets and shows superiority in dealing with extremely low-shot tasks.

References

[1] Y. Wang, Q. Yao, J. T. Kwok, and L. M. Ni, "Generalizing from a few examples: A survey on few-shot learning," *ACM computing surveys (csur)*, vol. 53, no. 3, pp. 1–34, 2020.

[2] B. M. Lake, R. Salakhutdinov, J. Gross, and J. B. Tenenbaum, "One shot learning of simple visual concepts," *Cognitive Science*, vol. 33, 2011.

[3] O. Vinyals, C. Blundell, T. Lillicrap, k. kavukcuoglu, and D. Wierstra, "Matching networks for one shot learning," in *Advances in Neural Information Processing Systems 29*, 2016, pp. 3630–3638.

[4] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 4077–4087.

[5] M. Ren, S. Ravi, E. Triantafillou, J. Snell, K. Swersky, J. B. Tenenbaum, H. Larochelle, and R. S. Zemel, "Meta-learning for semi-supervised few-shot classification," in *International Conference on Learning Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=HJcSzz-CZ

[6] W.-Y. Chen, Y.-C. Liu, Z. Kira, Y.-C. F. Wang, and J.-B. Huang, "A closer look at few-shot classification," in *International Conference on Learning Representations*, 2019.

[7] Y. Chen, X. Wang, Z. Liu, H. Xu, and T. Darrell, "A new meta-baseline for few-shot learning," *arXiv preprint arXiv:2003.04390*, 2020.

[8] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba, "Learning deep features for discriminative localization," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 2921–2929.

[9] S. Ravi and H. Larochelle, "Optimization as a model for few-shot learning," in *ICLR*, 2017.

[10] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proceedings of the 34th International Conference on Machine Learning*, vol. 70, 2017, pp. 1126–1135.

[11] Z. Li, F. Zhou, F. Chen, and H. Li, "Meta-SGD: Learning to learn quickly for few shot learning," *ArXiv*, vol. abs/1707.09835, 2017.

[12] Y. Lee and S. Choi, "Gradient-based meta-learning with learned layer-wise metric and subspace," in *ICML*, 2018.

[13] A. A. Rusu, D. Rao, J. Sygnowski, O. Vinyals, R. Pascanu, S. Osindero, and R. Hadsell, "Meta-learning with latent embedding optimization," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=BJgklhAcK7

[14] L. M. Zintgraf, K. Shiarlis, V. Kurin, K. Hofmann, and S. Whiteson, "Fast context adaptation via meta-learning," in *ICML*, 2019.

[15] S. Flennerhag, A. A. Rusu, R. Pascanu, F. Visin, H. Yin, and R. Hadsell, "Meta-learning with warped gradient descent," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=rkeiQlBFPB

[16] F. Sung, Y. Yang, L. Zhang, T. Xiang, P. H. S. Torr, and T. M. Hospedales, "Learning to compare: Relation network for few-shot learning," in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 1199–1208.

[17] B. Oreshkin, P. Rodríguez López, and A. Lacoste, "Tadam: Task dependent adaptive metric for improved few-shot learning," in *Advances in Neural Information Processing Systems 31*, 2018, pp. 721–731.

[18] C. Zhang, Y. Cai, G. Lin, and C. Shen, "Deepemd: Few-shot image classification with differentiable earth mover's distance and structured classifiers," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

[19] H.-J. Ye, H. Hu, D.-C. Zhan, and F. Sha, "Few-shot learning via embedding adaptation with set-to-set functions," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 8808–8817.

[20] C. Simon, P. Koniusz, R. Nock, and M. Harandi, "Adaptive subspaces for few-shot learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 4136–4145.

[21] N. Fei, Z. Lu, T. Xiang, and S. Huang, "MELR: Meta-learning via modeling episode-level relationships for few-shot learning," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=D3PcGLdMx0

[22] K. Lee, S. Maji, A. Ravichandran, and S. Soatto, "Meta-learning with differentiable convex optimization," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 10 657–10 665.

[23] S. Changpinyo, W.-L. Chao, B. Gong, and F. Sha, "Synthesized classifiers for zero-shot learning," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 5327–5336.

[24] Z. Liu, Z. Miao, X. Zhan, J. Wang, B. Gong, and S. X. Yu, "Large-scale long-tailed recognition in an open world," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 2537–2546.

[25] S. X. Hu, P. G. Moreno, Y. Xiao, X. Shen, G. Obozinski, N. Lawrence, and A. Damianou, "Empirical bayes transductive meta-learning with synthetic gradients," in *International Conference on Learning Representations*, 2020.

[26] Y. Liu, J. Lee, M. Park, S. Kim, E. Yang, S. Hwang, and Y. Yang, "Learning to propagate labels: Transductive propagation network for few-shot learning," in *International Conference on Learning Representations*, 2019.

[27] J. Zhao, Y. Yang, X. Lin, J. Yang, and L. He, "Looking wider for better adaptive representation in few-shot learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 12, pp. 10 981–10 989, May 2021.

[28] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[29] B. Li, J. Yan, W. Wu, Z. Zhu, and X. Hu, "High performance visual tracking with siamese region proposal network," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2018.

[30] Y. Wang, W.-L. Chao, K. Q. Weinberger, and L. van der Maaten, "Simpleshot: Revisiting nearest-neighbor classification for few-shot learning," *arXiv preprint arXiv:1911.04623*, 2019.

[31] S. Gidaris and N. Komodakis, "Dynamic few-shot visual learning without forgetting," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4367–4375.

[32] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein *et al.*, "Imagenet large scale visual recognition challenge," *International journal of computer vision*, vol. 115, no. 3, pp. 211–252, 2015.

[33] Y. Tian, Y. Wang, D. Krishnan, J. B. Tenenbaum, and P. Isola, "Rethinking few-shot image classification: a good embedding is all you need?" *arXiv preprint arXiv:2003.11539*, 2020.

[34] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.

[35] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019, pp. 8024–8035.

[36] A. Ravichandran, R. Bhotika, and S. Soatto, "Few-shot learning with embedded class models and shot-free meta training," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 331–339.

[37] R. Hou, H. Chang, M. Bingpeng, S. Shan, and X. Chen, "Cross attention network for few-shot classification," in *Advances in Neural Information Processing Systems*, 2019, pp. 4003–4014.

[38] Q. Sun, Y. Liu, T.-S. Chua, and B. Schiele, "Meta-transfer learning for few-shot learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 403–412.

[39] B. Liu, Y. Cao, Y. Lin, Q. Li, Z. Zhang, M. Long, and H. Hu, "Negative margin matters: Understanding margin in few-shot classification," in *European Conference on Computer Vision*. Springer, 2020, pp. 438–455.

[40] Y. Gao, N. Fei, G. Liu, Z. Lu, T. Xiang, and S. Huang, "Contrastive prototype learning with augmented embeddings for few-shot learning," *arXiv preprint arXiv:2101.09499*, 2021.

[41] M. N. Rizve, S. Khan, F. S. Khan, and M. Shah, "Exploring complementary strengths of invariant and equivariant representations for few-shot learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 10 836–10 846.

[42] M. Zhang, J. Zhang, Z. Lu, T. Xiang, M. Ding, and S. Huang, "IEPT: Instance-level and episode-level pretext tasks for few-shot learning," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=xzqLpqRzxLq

[43] L. v. d. Maaten and G. Hinton, "Visualizing data using t-sne," *Journal of Machine Learning research*, vol. 9, no. Nov, pp. 2579–2605, 2008.